

Arquitectura de bases de datos espaciales y estándares OGC

i Entorno de ejecución interactivo: Las consultas y sentencias DDL/DML de este capítulo asumen que la sesión de terminal dispone de las variables de conexión estándar de `libpq` debidamente configuradas. Si ejecuta los bloques de forma manual en la consola, verifique la asignación de parámetros descrita en la `?@sec-gestion-variables-entorno`.

Objetivos del capítulo

- Analizar los fundamentos conceptuales de las bases de datos y evaluar sus ventajas determinantes frente a los sistemas de archivos tradicionales.
- Comprender la arquitectura funcional, los niveles de abstracción y los subsistemas operativos de un sistema gestor de bases de datos (SGBD).
- Determinar el modelo de datos y la arquitectura de integración óptimos para soportar aplicaciones de sistemas de información geográfica (SIG).
- Examinar la implementación de los estándares internacionales del Open Geospatial Consortium (OGC) e ISO en motores relacionales extendidos.

De sistemas de archivos a bases de datos relacionales

El manejo de información en organizaciones y proyectos de geomática ha transitado históricamente desde esquemas basados en sistemas de archivos independientes hacia arquitecturas centralizadas administradas por **sistemas gestores de bases de datos (SGBD)** o **DBMS** (*Database Management System*).

Un sistema de archivos tradicional almacena los datos en estructuras cerradas dependientes de aplicaciones específicas (archivos planos, hojas de cálculo o formatos propietarios). Este enfoque presenta deficiencias estructurales críticas:

- **Redundancia e inconsistencia:** La duplicación descontrolada de archivos genera discrepancias en el estado de la información cuando se ejecutan actualizaciones parciales o no coordinadas.
- **Aislamiento y fragmentación de datos:** La dispersión de formatos y codificaciones heterogéneas impide la interoperabilidad y dificulta la formulación de consultas analíticas integradas.
- **Dependencia programa-datos:** Las aplicaciones se acoplan rígidamente a la estructura física de almacenamiento, lo que exige reescribir el software de consumo ante cualquier cambio en el formato de los archivos.

- **Vulnerabilidad transaccional y concurrencia deficiente:** Carencia de mecanismos ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad), lo que provoca condiciones de carrera, sobreescritura destructiva y corrupción de datos ante fallos del sistema o accesos simultáneos.
- **Costos elevados de mantenimiento y gobierno:** La ausencia de esquemas y catálogos normalizados fragmenta las políticas de seguridad, integridad y respaldo.

Frente a estas restricciones, una **base de datos** se define como una colección integrada, estructurada y no redundante de datos almacenados en soporte persistente, accesible concurrentemente por múltiples usuarios y aplicaciones mediante una estricta separación entre la representación lógica y la implementación física. Por su parte, el **SGBD** constituye la infraestructura de software responsable de proveer las interfaces y mecanismos para la definición, manipulación, optimización, persistencia y administración integral de dicha información.

Para unificar la nomenclatura técnica en este ámbito, se formalizan los siguientes conceptos fundamentales:

- **Dato:** Unidad mínima de registro simbólico (caracteres alfabéticos, numéricos o geospaciales brutos) carente de contexto inmediato.
- **Información:** Conjunto estructurado de datos procesados que adquiere significado dentro de un modelo conceptual o contexto analítico.
- **Atributo o campo:** Unidad atómica nominada que define una característica o propiedad de una entidad dentro de un dominio específico.
- **Tupla o registro:** Instancia o agregación ordenada de atributos que representa a una entidad concreta en un momento dado.
- **Relación o tabla:** Estructura lógica conformada por un conjunto no ordenado de tuplas consistentes con un esquema formal.
- **Esquema de base de datos:** Definición estructural y semántica de las relaciones, tipos de datos, restricciones y reglas de integridad, expresada mediante sentencias formales.
- **SGBD / DBMS (*Database Management System*):** Infraestructura de software responsable de proveer las interfaces y mecanismos para la definición, manipulación, optimización, persistencia y administración integral de las bases de datos. En la literatura técnica y en este texto se emplean indistintamente el acrónimo en español (**SGBD**) y la sigla internacional en inglés (**DBMS**).
- **SQL (*Structured Query Language*):** Estándar declarativo para la consulta, modificación y definición estructural de bases de datos relacionales.
 - **DDL (*Data Definition Language*):** Subconjunto de SQL orientado a la definición, alteración y eliminación de esquemas y objetos de la base de datos (**CREATE**, **ALTER**, **DROP**).
 - **DML (*Data Manipulation Language*):** Subconjunto de SQL orientado a la inserción, actualización, eliminación y recuperación de registros (**SELECT**, **INSERT**, **UPDATE**, **DELETE**).

- **Administrador de base de datos (DBA):** Rol técnico responsable del diseño físico, la optimización, las políticas de seguridad, el control de concurrencia y los planes de contingencia del motor.
- **Base de datos espacial (GeoDB):** Colección estructurada de datos espaciales y alfanuméricos persistidos bajo un esquema unificado, accesible de forma concurrente y gobernada por un catálogo de referencia cartográfica.
- **SGBD espacial / SDBMS (*Spatial Database Management System*):** Sistema gestor de bases de datos extendido con soporte nativo para tipos geométricos y geográficos, predicados topológicos, funciones analíticas multidimensionales y métodos de indexación espacial (árboles R / GiST). En diagramas arquitectónicos y documentación técnica se adopta comúnmente la sigla **SDBMS**.

En este texto se emplean de manera indistinta los términos **base de datos geográfica**, **base de datos espacial** o **GeoDB** para designar cualquier arquitectura de almacenamiento estructurado con soporte nativo de tipos espaciales, así como **SGBD espacial** y **SDBMS** para designar a su software gestor. Por convención, el término comercial **geodatabase** se reserva exclusivamente para las estructuras de almacenamiento propietarias del ecosistema ESRI.

Como se sintetiza en la Figure 1, el principio fundamental de una base de datos espacial radica en la **integración y centralización de esquemas**: el motor gestor consolida en un repositorio común atributos descriptivos, geometrías vectoriales tipadas y metadatos de referencia espacial, gobernados por un catálogo centralizado y un subsistema de integridad transaccional accesible por múltiples clientes en red.

Esta organización desacopla la representación lógica de la implementación física y resuelve las deficiencias de los sistemas de archivos:

- **Unicidad de la fuente:** Se elimina la redundancia derivada de mantener tablas de atributos dispersas en hojas de cálculo y geometrías en archivos vectoriales independientes.
- **Consistencia referencial síncrona:** Cualquier modificación sobre un registro (alfanumérico o espacial) queda sujeta a las restricciones del esquema global y a los mecanismos de aislamiento transaccional del motor.
- **Acceso concurrente estandarizado:** Distintos perfiles de usuario y capas de servicio consumen y modifican el mismo repositorio sin riesgo de bloqueo a nivel de archivo ni sobreescritura destructiva.

Componentes y funciones de un SGBD

Un sistema de gestión de bases de datos es el software especializado responsable de la definición, almacenamiento, mantenimiento, consulta y control transaccional de los datos. Su arquitectura desacopla el almacenamiento físico de la interfaz lógica del usuario para garantizar concurrencia, rendimiento e integridad de la información.

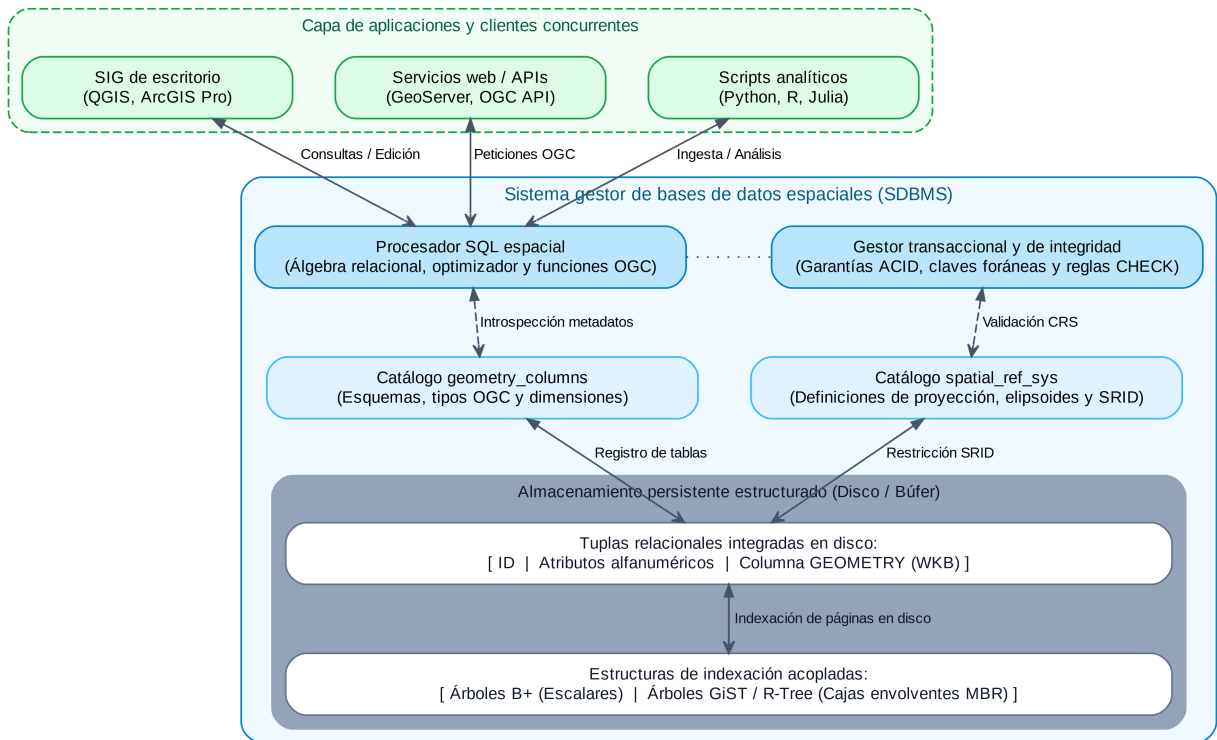


Figure 1: Centralización e integración de datos espaciales y alfanuméricos en un SDBMS.

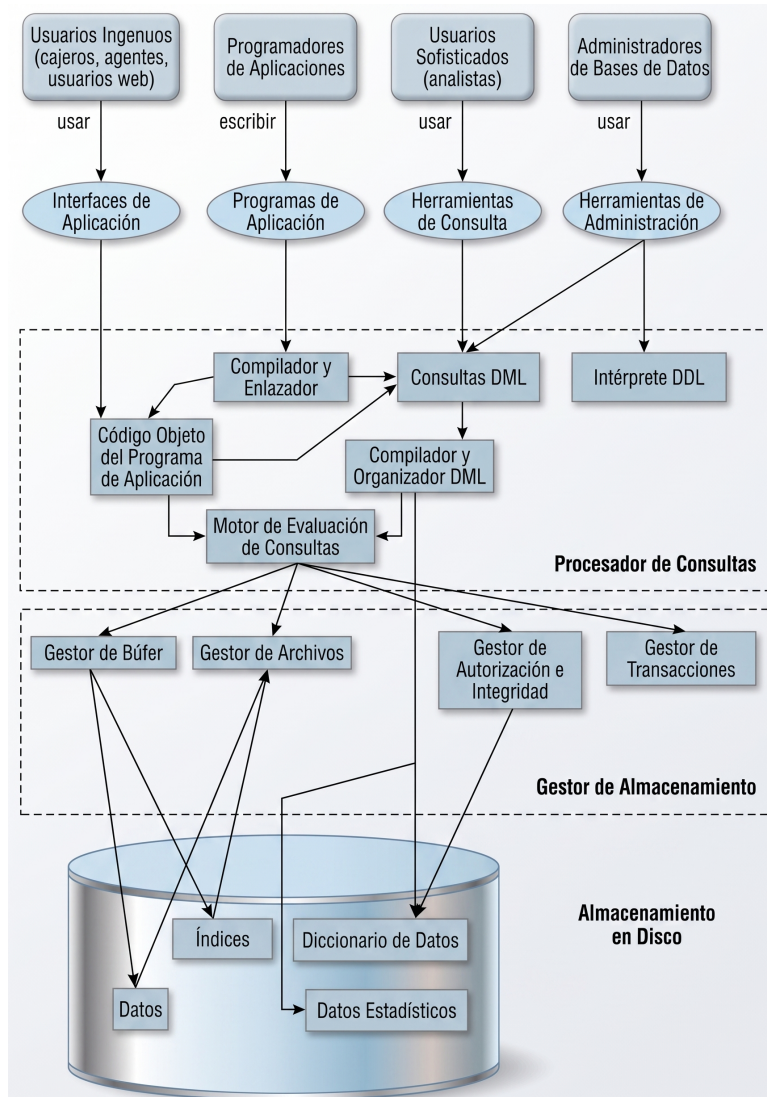


Figure 2: Componentes de un sistema de bases de datos, según Silberschatz, Korth y Sudarshan [-@silberschatz2002database].

En la Figure 2 se presenta la arquitectura clásica funcional de un SGBD estructurada en tres capas operativas fundamentales: el procesador de consultas, el gestor de almacenamiento y el almacenamiento en disco.

Procesador de consultas:

- **Intérprete DDL:** Procesa las sentencias de definición de datos y registra las definiciones correspondientes en el diccionario de datos.
- **Compilador y organizador DML:** Traduce las sentencias DML formuladas en lenguajes de consulta a un plan de evaluación de bajo nivel compuesto por operadores relacionales.
- **Motor de evaluación de consultas:** Ejecuta las instrucciones de bajo nivel generadas por el compilador DML interactuando directamente con el gestor de almacenamiento.
- **Compilador y enlazador de programas de aplicación:** Integra las llamadas a funciones DML embebidas en el código fuente con el motor de evaluación para generar código objeto ejecutable.

Gestor de almacenamiento:

- **Gestor de autorización e integridad:** Valida las restricciones de integridad declarativas y comprueba los privilegios de acceso de los usuarios a partir de los datos estadísticos y el diccionario.
- **Gestor de transacciones:** Garantiza que la base de datos permanezca en un estado consistente a pesar de fallos del sistema, coordinando la ejecución concurrente sin interferencias.
- **Gestor de archivos:** Asigna y gestiona el espacio en el almacenamiento secundario y las estructuras de datos usadas para representar los registros.
- **Gestor de búfer:** Controla la transferencia de páginas entre la memoria volátil intermedia (*buffer pool*) y el almacenamiento en disco persistente.

Estructuras de almacenamiento en disco:

- **Datos:** Bloques físicos que alojan los registros de la base de datos.
- **Índices:** Estructuras de acceso rápido para acelerar la localización de tuplas específicas.
- **Diccionario de datos:** Almacén centralizado de metadatos sobre la estructura lógica y esquemas del sistema.
- **Datos estadísticos:** Información cuantitativa del catálogo sobre la distribución de las relaciones, empleada por el optimizador de consultas.

En la Table 1 se detalla la correspondencia entre los módulos de la arquitectura de Silberschatz, su nivel operativo y su responsabilidad fundamental dentro del motor de base de datos.

Table 1: Componentes principales del SGBD según Silberschatz y sus responsabilidades funcionales.

Componente	Nivel funcional	Responsabilidad principal
Intérprete DDL	Procesamiento lógico	Procesamiento de sentencias de definición y actualización de metadatos.
Compilador y organizador DML	Procesamiento lógico	Traducción de consultas de alto nivel a planes de ejecución evaluables.
Motor de evaluación de consultas	Ejecución lógica	Orquestación y despacho de instrucciones de bajo nivel al motor de almacenamiento.
Gestor de autorización e integridad	Almacenamiento / Control	Validación de restricciones declarativas y verificación de permisos.
Gestor de transacciones	Transaccional	Preservación de propiedades ACID y control de operaciones simultáneas.
Gestor de búfer	Almacenamiento físico	Paginación y caché de bloques de memoria entre disco y RAM.
Gestor de archivos	Almacenamiento físico	Asignación de espacio físico y direccionamiento de bloques de registros e índices.

Como marco comparativo, Elmasri y Navathe [Elmasri2007fundamentos] plantean una descomposición modular con mayor detalle en los subsistemas de ejecución de transacciones y el procesador de base de datos en tiempo de ejecución (*runtime database processor*), tal como se ilustra en la Figure 3.

Importancia y características de los SGBD

En la infraestructura tecnológica contemporánea, los SGBD constituyen el núcleo operativo para la persistencia, disponibilidad e integridad de los activos de información. Su diseño minimiza la fragilidad intrínseca de los sistemas de archivos mediante la implementación de niveles formales de abstracción de datos (esquema físico, lógico y de vistas), permitiendo aislar las aplicaciones cliente de las modificaciones en el almacenamiento subyacente.

Las propiedades funcionales y arquitectónicas que distinguen a un SGBD frente a soluciones ad-hoc comprenden:

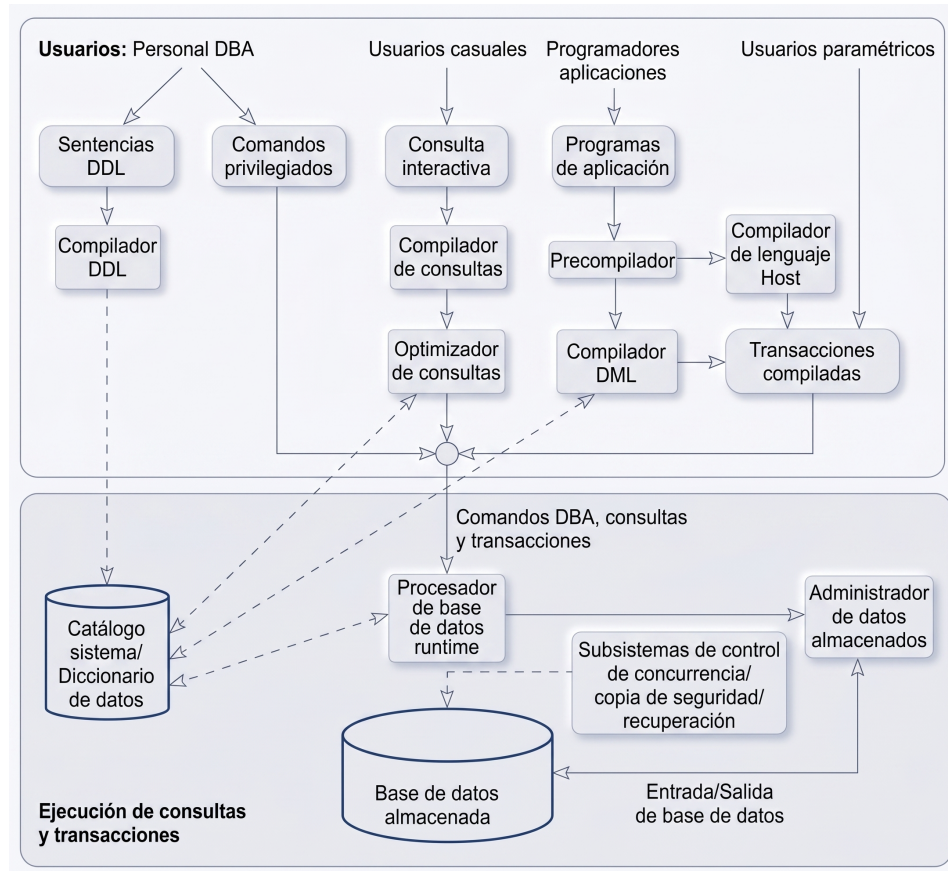


Figure 3: Arquitectura de componentes y ejecución de transacciones, según Elmasri y Navathe [-@ramez2007fundamentos].

- **Independencia lógica y física:** Capacidad de modificar la organización física de los archivos o el diseño conceptual de las tablas sin requerir la reescritura de las aplicaciones que consumen la base de datos.
- **Soporte de tipado abstracto y dominios complejos:** Definición rigurosa de tipos primitivos (escalares, numéricos, temporales) y estructurados (JSON, tipos geométricos/geográficos OGC, grandes objetos binarios o BLOB).
- **Procesamiento declarativo y optimización de consultas:** Interpretación de sentencias en SQL estándar mediante un optimizador basado en costos (CBO), el cual formula planes de ejecución eficientes a partir de la distribución estadística de los datos.
- **Mecanismos de concurrencia y garantías ACID:** Aislamiento de transacciones concurrentes para evitar anomalías como lecturas sucias, actualizaciones perdidas y lecturas no repetibles.
- **Estructuras de acceso optimizado:** Implementación de índices univariados y multi-dimensionales (árboles B+, GiST, SP-GiST, R-Trees) que reducen exponencialmente los costos de entrada/salida (I/O) en operaciones de filtrado y agregación.
- **Tolerancia a fallos y recuperación:** Persistencia transaccional apoyada en registros de auditoría y bitácoras de operaciones (*Write-Ahead Logging*), asegurando la reconstrucción del estado consistente del sistema ante interrupciones no programadas de energía o caídas del kernel.
- **Seguridad y auditoría granular:** Control de accesos basado en roles (RBAC), mecanismos de autenticación criptográfica, cifrado en reposo y en tránsito, y políticas de seguridad a nivel de fila (*Row-Level Security*).

Perfiles de usuario y responsabilidades del administrador

La arquitectura canónica formaliza cuatro categorías de usuarios en función de su nivel de interacción técnica y el grado de abstracción con el que operan:

- **Usuarios ingenuos o paramétricos:** Interactúan con el sistema de forma indirecta mediante interfaces de usuario precompiladas y programas de aplicación (formularios web, aplicaciones móviles o terminales transaccionales), ejecutando operaciones predefinidas sin conocer la sintaxis SQL ni el modelo de datos subyacente.
- **Programadores de aplicaciones:** Desarrollan artefactos de software en lenguajes anfitriones (Python, Java, C++, Julia), embebiendo sentencias DML o invocando interfaces de programación (ODBC, JDBC, APIs nativas) para conectar las aplicaciones con el motor de base de datos.
- **Usuarios sofisticados o analistas:** Interactúan directamente con el procesador de consultas mediante herramientas interactivas, consolas SQL o plataformas de ciencia de datos, formulando consultas analíticas complejas y extrayendo conjuntos de datos sin intermediación de código de aplicación estático.

- **Usuarios especializados:** Diseñan e implementan sistemas de bases de datos no tradicionales, tales como motores de bases de conocimiento, procesadores analíticos multidimensionales o sistemas de información espacial avanzados.

Dentro de este ecosistema, el **administrador de bases de datos (DBA)** representa la figura de control centralizado encargada del ciclo de vida operativo, el rendimiento y la seguridad del sistema gestor. Sus responsabilidades comprenden:

- **Definición del esquema conceptual y físico:** Redacción de especificaciones en lenguaje DDL para la creación del modelo relacional, las cuales el compilador DDL interpreta y almacena en el diccionario de datos.
- **Selección de métodos de acceso y estructuras de almacenamiento:** Definición de la topología de almacenamiento físico, asignación de espacios de tablas (*tablespaces*), tamaño de páginas de disco y diseño de índices primarios y secundarios.
- **Modificación y afinamiento físico (*tuning*):** Ajuste dinámico de parámetros del gestor de búferes, memoria intermedia y reestructuración periódica de índices para mitigar cuellos de botella según el comportamiento de la carga de trabajo.
- **Gestión de autorizaciones y directivas de seguridad:** Creación de roles de usuario, asignación granular de privilegios sobre esquemas o tablas, y administración de credenciales bajo el principio de mínimo privilegio.
- **Supervisión de restricciones de integridad:** Especificación de reglas declarativas (claves primarias, integridad referencial, restricciones de dominio) para que el gestor de integridad impida estados inconsistentes.
- **Mantenimiento preventivo, respaldo y contingencia:** Definición de políticas de copias de seguridad (completas, incrementales y continuas), monitorización del almacenamiento secundario y ejecución periódica de simulacros de recuperación ante desastres.

Niveles de abstracción e independencia de datos

Un sistema gestor de bases de datos abstrae la complejidad del almacenamiento secundario para presentar una representación simplificada de la información. La arquitectura estándar de tres niveles (ANSI/SPARC) formaliza este principio mediante capas operativas independientes, como se ilustra en la [Figure 4](#).

Las tres capas estructuran responsabilidades específicas:

1. **Nivel físico o interno:** Describe la organización técnica en que los registros se estructuran en el disco: distribución de páginas de datos, métodos de compresión, rutas de acceso físicas y estructuras de indexación secundarias (árboles B+ o árboles R/GiST en contextos espaciales).
2. **Nivel conceptual o lógico:** Modela la estructura integral del dominio de datos con independencia de los detalles del hardware de almacenamiento. En este nivel se definen las entidades, atributos, dominios (incluyendo tipos geométricos OGC), restricciones de

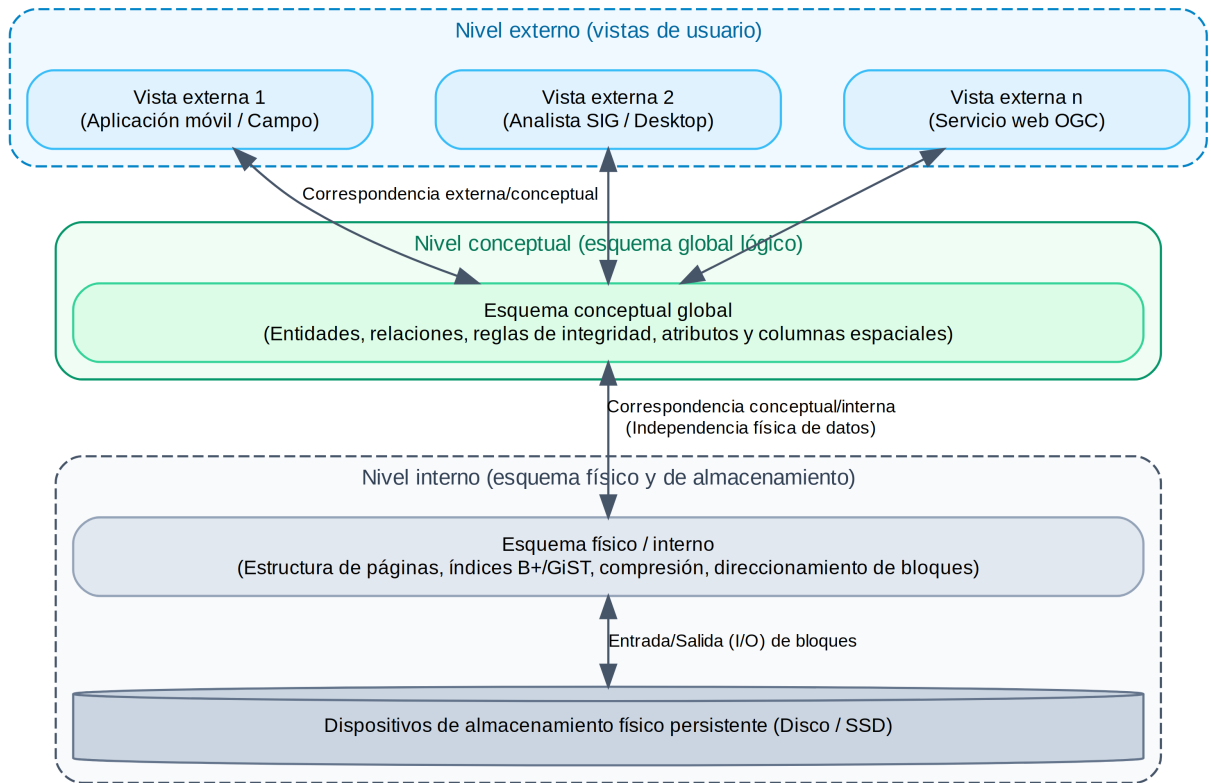


Figure 4: Arquitectura ANSI/SPARC de tres niveles y correspondencias de independencia de datos.

integridad (**CHECK**, claves primarias, llaves foráneas) y relaciones lógicas entre conjuntos de datos.

3. **Nivel de visión o externo:** Define representaciones parciales y personalizadas del esquema lógico orientadas a distintos perfiles de usuario, roles operativos o módulos de software mediante vistas relacionales (**VIEW**) o políticas de seguridad a nivel de filas y columnas.

En la Table 2 se sintetizan las responsabilidades de cada estrato arquitectónico, los perfiles de usuario involucrados y su manifestación técnica en motores geoespaciales.

Table 2: Niveles de abstracción arquitectónica y su instanciación técnica.

Nivel de abstracción	Audiencia principal	Mecanismos de implementación	Instanciación espacial (SGBD espacial)
Visión (externo)	Usuarios finales y analistas	Vistas SQL (VIEW , MATERIALIZED VIEW), permisos de rol.	Capas filtradas por cobertura territorial o temática.
Conceptual (lógico)	Administrador de datos y diseñador	Esquemas DDL, restricciones relacionales, tipos de datos.	Tipos de datos espaciales (GEOMETRY , GEOGRAPHY , SRID).
Físico (interno)	DBA y motor de almacenamiento	Asignación de páginas en disco, bloques físicos, WAL.	Codificación binaria (WKB) e índices espaciales (GiST, SP-GiST).

La separación desacoplada entre estos estratos garantiza dos propiedades operativas fundamentales:

- **Independencia física de datos:** Capacidad de reestructurar la disposición de almacenamiento, cambiar de medio físico, modificar el particionamiento de tablas o crear nuevos índices en disco sin requerir ajustes en el esquema conceptual ni reescribir las sentencias de consulta de las aplicaciones cliente.
- **Independencia lógica de datos:** Capacidad de expandir o modificar el esquema conceptual (añadir nuevas columnas, entidades o relocalizar tablas) sin alterar las aplicaciones que consumen vistas externas preexistentes, siempre que estas no dependan de los elementos modificados.

Evolución de modelos de datos y la dimensión espacial

A lo largo del desarrollo de los sistemas de información, los modelos de bases de datos se han estructurado en torno a paradigmas que responden tanto a las limitaciones del hardware de cada época como a la complejidad semántica de los datos:

- **Modelos basados en registros y punteros físicos (Tipo 1):** Dominantes en las décadas de 1950 y 1960 (modelos jerárquicos y de red), donde las asociaciones entre entidades se implementaban mediante enlaces físicos de memoria o apuntadores en disco.
- **Modelo relacional (Tipo 2):** Introducido por E. F. Codd en 1970, fundamentado en la teoría de conjuntos y la lógica de predicados de primer orden. La información se modela en relaciones (tablas compuestas por tuplas y atributos atómicos), y las vinculaciones se realizan exclusivamente a través de valores escalares compartidos (claves primarias y foráneas).
- **Modelos objeto-relacionales y orientados a objetos (Tipo 3):** Formulados a partir de 1990 para superar la incapacidad del modelo relacional plano al manejar datos no atómicos. Introducen encapsulamiento, herencia, métodos y soporte nativo para tipos abstractos de datos (ADT), habilitando la persistencia e indexación de geometrías espaciales y tipos de datos multidimensionales.
- **Modelos no relacionales y de grafos (NoSQL):** Consolidados a partir de 2010 para resolver cuellos de botella de escalabilidad horizontal y rigidez de esquemas. Reemplazan la normalización formal por esquemas flexibles de lectura (*schema-on-read*) divididos en cuatro familias: documentales (JSON/BSON), clave-valor, matrices de columnas anchas y grafos de propiedades, estos últimos formalizados mediante el estándar internacional ISO/IEC 39075:2024 (GQL).
- **Evolución contemporánea (NewSQL, multimodelo y bases de datos vectoriales):** Desde 2012, el modelo relacional tradicional derivó hacia sistemas distribuidos elásticos con consistencia transaccional estricta mediante algoritmos de consenso (NewSQL). De forma paralela, surgieron motores convergentes multimodelo y bases de datos vectoriales optimizadas para búsqueda de similitud semántica mediante grafos de mundo pequeño (*HNSW*) sobre representaciones densas de alta dimensionalidad (*embeddings*).

En la Figure 5 se resume la trayectoria evolutiva de estos paradigmas, destacando la bifurcación entre sistemas orientados a transacciones ACID y soluciones diseñadas para escalabilidad distribuida o cómputo multidimensional.

En la Table 3 se sistematizan los motores de bases de datos representativos de la disciplina, estructurados conforme a los paradigmas arquitectónicos presentados.

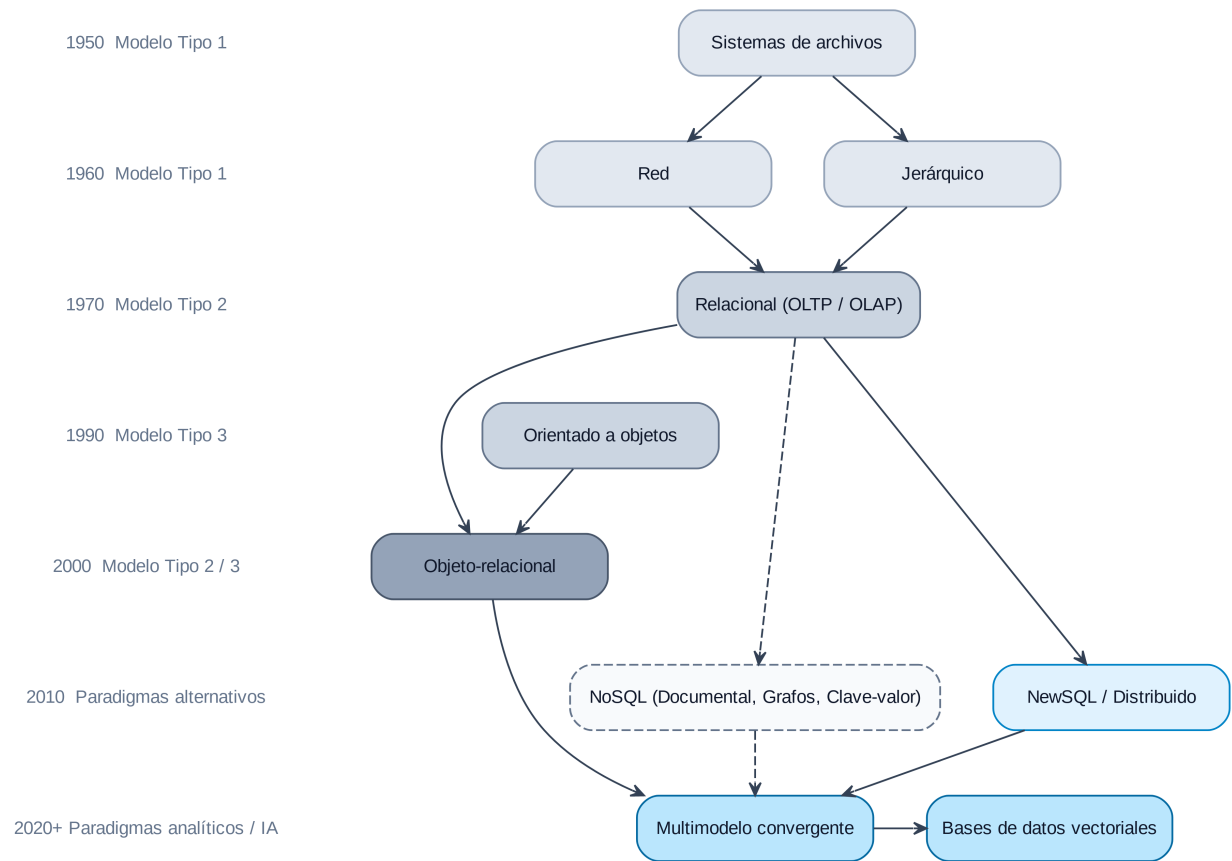


Figure 5: Cronología de modelos de bases de datos con alineación temporal formal.

Table 3: Clasificación de sistemas gestores de bases de datos según su modelo paradigmático.

Modelo arquitectónico	Época / Taxonomía	Sistemas y motores representativos	Mecanismo de estructuración predominante
Sistemas de archivos y métodos de acceso	Década de 1950 (Tipo 1)	IBM ISAM, VSAM, archivos secuenciales	Registros mapeados por <i>offset</i> físico en disco o cinta.
Modelo jerárquico	Década de 1960 (Tipo 1)	IBM IMS (<i>Information Management System</i>)	Estructuras en árbol con relaciones 1:N mediante punteros físicos padre-hijo.
Modelo en red	Década de 1960 (Tipo 1)	IDS, Cullinet IDMS (estándar CODASYL)	Grafos de registros con relaciones N:M gestionadas mediante conjuntos (<i>sets</i>) y apuntadores en anillo.
Modelo relacional transaccional (OLTP)	Década de 1970 (Tipo 2)	IBM System R, Ingres, IBM DB2, MySQL, SQLite	Tablas estructuradas por filas con tuplas y atributos atómicos, gobernadas por álgebra relacional y cumplimiento ACID.
Modelo relacional analítico columnar (OLAP)	Década de 2010 (2019+)	DuckDB, ClickHouse, MonetDB	Álgebra relacional y sintaxis SQL estándar implementada sobre almacenamiento columnar y ejecución vectorial en memoria/CPU (SIMD).
Orientado a objetos (OODBMS)	Década de 1990 (Tipo 3)	ObjectStore, db4o, Versant, Objectivity/DB	Persistencia directa de instancias del lenguaje anfitrión con encapsulamiento, identidad propia (OID) y métodos.

Modelo arquitectónico	Época / Taxonomía	Sistemas y motores representativos	Mecanismo de estructuración predominante
Objeto-relacional (ORDBMS)	Década de 2000 (Tipo 2/3)	PostgreSQL (+ PostGIS), Oracle (v8i+), IBM Informix	Núcleo relacional con soporte nativo de tipos abstractos (ADT), polimorfismo, herencia de tablas e índices extensibles (GiST/SP-GiST).
NoSQL: Documental	Década de 2010	MongoDB, Apache CouchDB	Registros semiestructurados jerárquicos (JSON, BSON) con esquema dinámico (<i>schema-on-read</i>).
NoSQL: Grafos de propiedades	Década de 2010	Neo4j, Amazon Neptune, Memgraph	Nodos, aristas y propiedades tipadas optimizados para recorridos de relaciones complejas sin operaciones de combinación (<i>joins</i>).
NoSQL: Clave-valor	Década de 2010	Redis, Amazon DynamoDB, Memcached	Almacén asociativo indexado por <i>hash</i> unidimensional de muy baja latencia en memoria o disco.
NoSQL: Familias de columnas (<i>wide-column</i>)	Década de 2010	Apache Cassandra, Apache HBase, ScyllaDB	Tablas multidimensionales dispersas y particionadas horizontalmente, optimizadas para escrituras masivas distribuidas.

Modelo arquitectónico	Época / Taxonomía	Sistemas y motores representativos	Mecanismo de estructuración predominante
NewSQL / SQL distribuido	Década de 2010 (2012+)	Google Cloud Spanner, CockroachDB, YugabyteDB, TiDB	Álgebra relacional estándar y transaccionalidad ACID global sobre clústeres elásticos mediante protocolos de consenso (Paxos/Raft).
Multimodelo nativo	Década de 2010 (2015+)	ArangoDB, Microsoft Azure Cosmos DB	Motores concebidos de origen para procesar de forma unificada grafos, documentos, clave-valor y estructuras tabulares.
Bases de datos vectoriales	Década de 2020 (2020+)	Milvus, Qdrant, Pinecone, Weaviate, Chroma	Almacenamiento e indexación de <i>embeddings</i> multidimensionales mediante grafos HNSW o cuantización vectorial para cálculo de distancias métricas.

La dimensión espacial como catalizador del modelo objeto-relacional

La adopción de modelos de bases de datos en geomática estuvo condicionada por la complejidad intrínseca de los datos geográficos frente a los datos alfanuméricos tradicionales. Esta complejidad se manifiesta en cuatro dimensiones críticas:

- **Multidimensionalidad:** Las entidades espaciales poseen extensión geométrica en dos, tres o cuatro dimensiones (coordenadas X , Y , elevación Z , y valores de medición lineal M).

- **Topología y continuidad:** Los objetos en el territorio exhiben relaciones espaciales continuas (adyacencia, contención, solapamiento, conectividad) que no se reducen a operadores de comparación escalar ($=$, $<$, $>$).
- **Tipologías heterogéneas:** Coexisten entidades vectoriales discretas (puntos, líneas, polígonos) y campos continuos (matrices ráster y mallas irregulares).
- **Ineficiencia de árboles de búsqueda unidimensionales:** Los árboles B y B+ son incapaces de indexar eficazmente cajas envolventes mínimas (*Bounding Boxes*) o rangos multidimensionales.

Como se sintetiza en la Figure 6, la evolución arquitectónica respondió a la necesidad de gestionar simultáneamente atributos escalares simples y geometrías dotadas de comportamiento topológico.

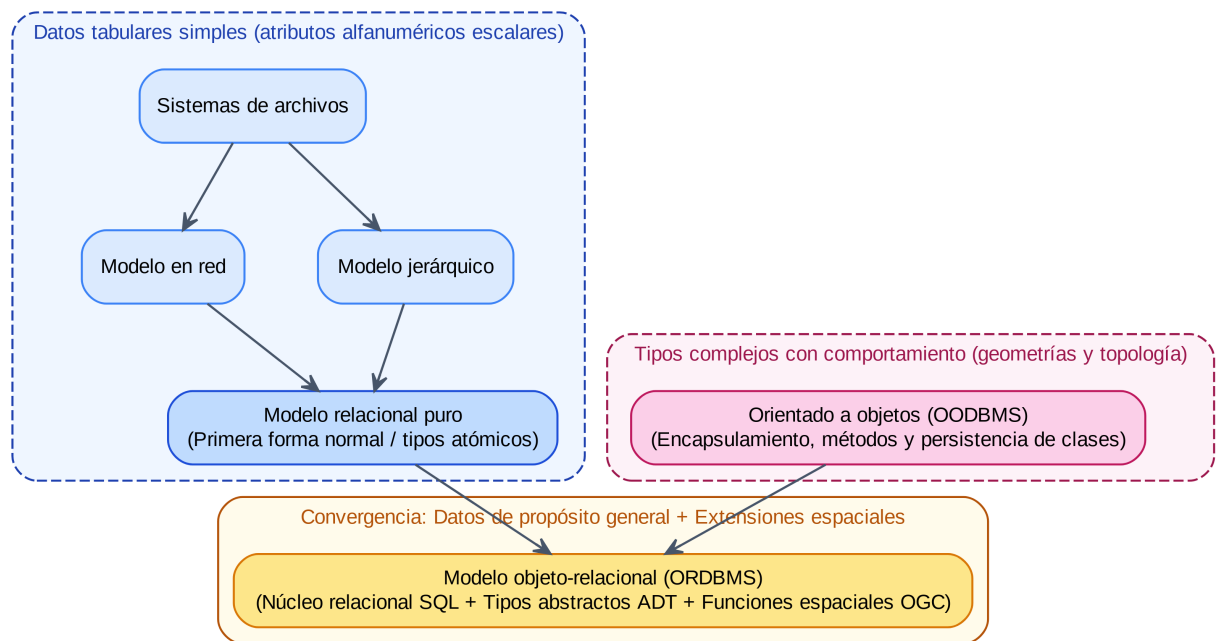


Figure 6: Convergencia de requerimientos de datos hacia el modelo objeto-relacional.

El modelo relacional tradicional garantizaba integridad referencial, consistencia transaccional y optimización sobre atributos atómicos escalares, pero carecía de mecanismos nativos para operar geometrías complejas sin desarticularlas en colecciones de coordenadas normalizadas. Por su parte, los sistemas orientados a objetos (OODBMS) permitían modelar geometrías y operadores topológicos como métodos de clase, pero sacrificaban la madurez del álgebra relacional, el lenguaje declarativo SQL y la eficiencia de los optimizadores basados en costos.

El modelo objeto-relacional resolvió esta encrucijada al extender el motor relacional con tipos de datos abstractos (ADT), índices espaciales multidimensionales (árboles R sobre infraestruc-

tura GiST) y predicados topológicos evaluables dentro del planificador de consultas.

Consideraciones arquitectónicas: extensibilidad y motores de cómputo analítico

Al analizar el ecosistema de datos espaciales y analíticos, es necesario distinguir entre la arquitectura de núcleo de un SGBD, los mecanismos de convergencia multimodelo por extensibilidad y los motores de procesamiento distribuido sobre lagos de datos:

- **Estatus arquitectónico de PostgreSQL y su convergencia multimodelo:** PostgreSQL es en su diseño de núcleo un sistema objeto-relacional (ORDBMS). La extensibilidad provista por su arquitectura permitió implementar soporte para datos semiestructurados (JSONB), tipos espaciales avanzados e índices multidimensionales mediante PostGIS, y búsqueda por similitud vectorial mediante `pgvector`. De este modo, aunque su fundamento teórico no es multimodelo nativo, opera en la práctica como un sustrato convergente para cargas de trabajo heterogéneas.
- **Diferenciación funcional de DuckDB:** DuckDB no plantea un nuevo modelo algebraico de datos, sino una reconfiguración de la arquitectura de ejecución. Se clasifica dentro del modelo relacional (Tipo 2), pero desacopla el paradigma tradicional de almacenamiento orientado a filas (*row-oriented*) para utilizar almacenamiento columnar vectorial en memoria. Su modelo de despliegue embebido (*in-process*) elimina la sobrecarga de red del modelo cliente-servidor, convirtiéndose en un motor OLAP especializado para análisis local sobre grandes volúmenes de datos tabulares y geoespaciales.
- **Distinción entre SGBD y motores de cómputo analítico (Apache Sedona):** Soluciones como Apache Sedona (anteriormente GeoSpark) no constituyen sistemas gestores de bases de datos (SGBD). Carecen de motores de almacenamiento persistente propios, gestores de transacciones ACID, subsistemas de bitácora (*Write-Ahead Logging*) y gestores de búfer en disco. Sedona es un **motor de procesamiento espacial distribuido** que actúa como capa analítica sobre entornos de cómputo en clúster (Apache Spark y Apache Flink), permitiendo evaluar predicados espaciales e indexación en memoria sobre archivos externos (GeoParquet, Shapefiles o capas de lagos de datos).

Arquitecturas para aplicaciones de sistemas de información geográfica

Históricamente, la vinculación entre los sistemas de información geográfica (SIG) y los sistemas de gestión de datos ha transitado por tres generaciones arquitectónicas principales, caracterizadas por el nivel de acoplamiento entre la geometría, los atributos temáticos y el motor de ejecución.

En la [Figure 7](#) se ilustra la evolución conceptual desde esquemas propietarios aislados hasta la centralización transaccional en motores espaciales integrados.

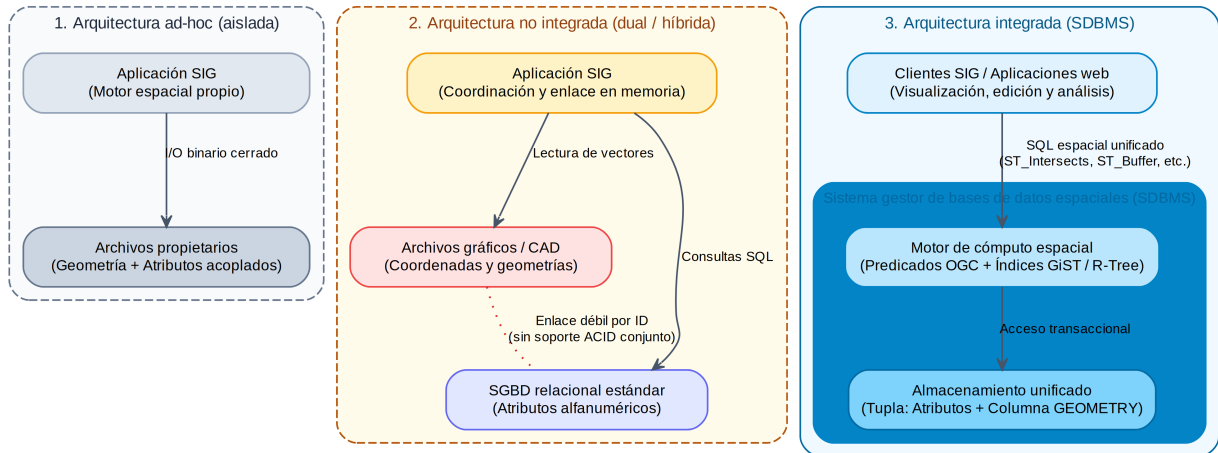


Figure 7: Evolución de las arquitecturas de integración entre SIG y bases de datos.

Arquitectura ad-hoc

En esta configuración inicial, los paquetes de software SIG implementan formatos binarios cerrados y algoritmos propietarios de cálculo geométrico ejecutados directamente sobre el sistema de archivos:

- **Ventajas:** Alto rendimiento y eficiencia computacional para operaciones cartográficas locales en estaciones monousuario, al carecer de sobrecarga (*overhead*) por capas de red o bloqueos transaccionales.
- **Limitaciones:** Nula extensibilidad hacia nuevas aplicaciones, imposibilidad de compartir datos fuera del entorno de origen, ausencia de control de concurrencia y proliferación de información redundante e inconsistente.

Arquitectura no integrada (dual o híbrida)

Este esquema desacopla el almacenamiento en dos repositorios independientes: los atributos temáticos se delegan a un SGBD relacional estándar, mientras que las geometrías continúan gestionándose mediante el sistema de archivos tradicional:

- **Ventajas:** Incorpora las capacidades analíticas, de seguridad y de consulta declarativa del SGBD para la fracción alfanumérica de la información.
- **Limitaciones:** La responsabilidad de integrar y sincronizar la geometría con la tabla recae exclusivamente en la capa de aplicación (*application programs*), lo que genera un rendimiento inferior al de las soluciones ad-hoc, sobrecarga de memoria en el cliente y riesgo permanente de inconsistencia transaccional (huérfanos espaciales o tablas sin geometría asociada).

Arquitectura integrada (SDBMS)

Representa el estándar contemporáneo, donde el sistema gestor de bases de datos asume tanto la geometría como los atributos descriptivos dentro de un mismo motor estructurado:

- **Reutilización y extensibilidad:** Desacopla los datos espaciales de los programas SIG clientes, permitiendo que múltiples aplicaciones heterogéneas (escritorio, servicios web OGC, pipelines en Python/R/Julia) consuman y actualicen las mismas capas de forma concurrente.
- **Procesamiento en el servidor:** Las funciones analíticas, el cálculo de relaciones topológicas y la indexación multidimensional se ejecutan directamente en el motor de la base de datos, optimizando el ancho de banda al retornar únicamente el subconjunto filtrado.
- **Compromiso de rendimiento (*trade-off*):** Aunque puede presentar un costo operativo marginalmente superior frente a algoritmos locales ad-hoc debido a la gestión de transacciones ACID y la serialización SQL, dicho margen queda plenamente compensado por la integridad estructural, la escalabilidad y la interoperabilidad institucional de los datos.

Distribución de responsabilidades entre el SIG y el SGBD objeto-relacional

La adopción de una arquitectura integrada formaliza un esquema multicapa donde se dividen estrictamente las tareas entre la aplicación cliente y el motor gestor de bases de datos, tal como se esquematiza en la Figure 8.

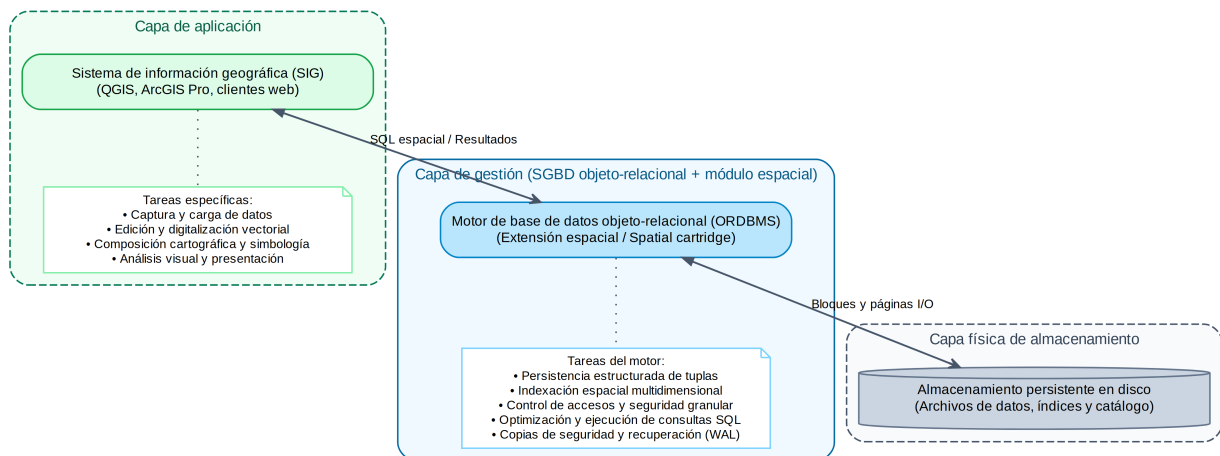


Figure 8: Distribución de capas y responsabilidades entre el software SIG y el SGBD objeto-relacional.

Bajo este modelo de distribución operativa se derivan ventajas determinantes:

- **Descarga computacional de la aplicación cliente:** Al incorporar un **módulo espacial** (*spatial extension* o *cartridge*) directamente acoplado al núcleo del motor, los algoritmos geométricos computacionalmente intensivos (cálculo de áreas, uniones espaciales, generación de zonas de influencia o evaluación de relaciones topológicas) se procesan dentro del servidor.
- **Unificación de datos convencionales y espaciales:** El SGBD administra atributos alfanuméricos y objetos geométricos bajo un mismo esquema declarativo, garantizando que las reglas de integridad referencial, directivas de seguridad y restricciones de dominio apliquen uniformemente sobre toda la entidad.
- **Simplificación del ciclo de vida del software:** Las aplicaciones SIG se liberan de la responsabilidad de implementar motores de indexación propios o controladores de concurrencia a nivel de archivo. Esto reduce la complejidad del código cliente y previene inconsistencias por accesos simultáneos.

PostgreSQL y la extensión PostGIS

PostgreSQL es un sistema gestor de bases de datos objeto-relacional (ORDBMS) de código abierto, diseñado con una arquitectura extensible que permite incorporar nuevos tipos de datos, funciones definidas por el usuario en múltiples lenguajes (C, PL/pgSQL, Python) y métodos de acceso indexados especializados.

PostGIS es la extensión espacial oficial de PostgreSQL. Transforma el motor relacional en un SDBMS completo al incorporar:

- Tipos de datos espaciales geométricos vectoriales (**geometry**), geodésicos (**geography**), matriciales (**raster**) y estructuras topológicas (**topology**).
- Funciones de análisis espacial conformes a las especificaciones internacionales de la OGC.
- Soporte para métodos de indexación espacial mediante árboles de búsqueda generalizada (GiST, *Generalized Search Tree*) y árboles R.

i Instrucciones específicas para el sistema anfitrión (Windows)

Cuando se administra PostgreSQL de forma nativa sobre el sistema anfitrión Windows, el servicio del clúster puede gestionarse desde la consola de servicios (**services.msc**), identificando la entrada **postgresql-x64-<version>**. Alternativamente, desde una terminal de PowerShell con privilegios administrativos, se puede consultar el estado del servicio mediante:

```
Get-Service -Name postgresql*
```

Para asegurar que las utilidades de línea de comandos (**psql**, **pg_dump**, **pg_restore**) respondan globalmente en la consola del sistema anfitrión, la ruta binaria del motor (por ejemplo, **C:\Program Files\PostgreSQL\<version>\bin**) debe estar registrada en la

variable de entorno del sistema Path.

Verificación de PostGIS en el sistema anfitrión (Windows)

Para comprobar que el entorno espacial se encuentra habilitado y operativo en la base de datos de trabajo `sig_db_unal`, se consulta la firma completa de dependencias compiladas contra el motor relacional:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

# Inspección de versión y librerías geoespaciales vinculadas a PostGIS
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-x -P format=wrapped -P columns=90 \
-c "SELECT postgis_full_version();"

-[ RECORD 1 ]-----+-----
postgis_full_version | POSTGIS="3.6.2 3.6.2" [EXTENSION] PGSQL="180" GEOS="3.14.1dev-CAPI.
                        | .-1.20.4" PROJ="8.2.1 NETWORK_ENABLED=OFF URL_ENDPOINT=https://cdn..
                        | .proj.org USER_WRITABLE_DIRECTORY=C:\WINDOWS\ServiceProfiles\Networ.
                        | .kService\AppData\Local\proj DATABASE_PATH=D:\Programs\PostgreSQL\1.
                        | .8\share\contrib\postgis-3.6\proj\proj.db" (compiled against PROJ 8.
                        | ..2.1) LIBXML="2.12.5" LIBJSON="0.12" LIBPROTOBUF="1.2.1" WAGYU="0..
                        | .5.0 (Internal)"
```

El resultado de esta función expone las librerías base sobre las cuales opera el motor espacial:

- **GEOS (*Geometry Engine Open Source*)**: Provee las primitivas de cálculo computacional y evaluación de predicados topológicos.
- **PROJ**: Motor de transformación y reproyección entre sistemas de referencia de coordenadas.
- **GDAL (*Geospatial Data Abstraction Library*)**: Librería de traducción y soporte para formatos ráster y vectoriales.
- **LIBXML2**: Procesador de serializaciones XML/GML.

Estándar OGC Simple Features for SQL (ISO 19125)

El Open Geospatial Consortium (OGC) y la Organización Internacional de Normalización (ISO/TC 211) formularon la especificación *Simple Features for SQL* (SFS), recogida formalmente bajo la norma ISO 19125. Esta especificación define una arquitectura estandarizada para almacenar, manipular y consultar geometrías 2D en bases de datos relacionales.

Jerarquía de tipos geométricos OGC

El estándar define una jerarquía orientada a objetos donde la clase raíz abstracta es **Geometry**. En la Figure 9 se ilustra el árbol de herencia para las primitivas simples y las agrupaciones espaciales.

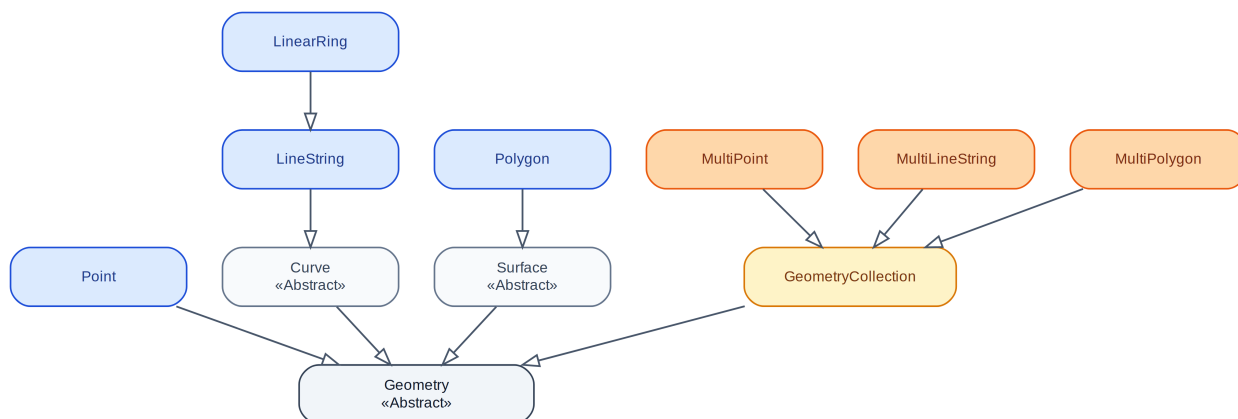


Figure 9: Jerarquía de tipos de datos geométricos según la norma OGC SFS / ISO 19125.

En la Table 4 se detallan las características topológicas y dimensionales de los tipos definidos en la norma.

Table 4: Primitivas y colecciones geométricas del estándar OGC.

Tipo geométrico	Dimensión topológica	Descripción formal OGC
Point	0	Posición geométrica aislada descrita por un único par ordenado (X, Y) .
LineString	1	Curva lineal interpolada entre dos o más vértices.
LinearRing	1	LineString cerrada y no autosecante (define límites poligonales).
Polygon	2	Superficie planar delimitada por un anillo exterior y cero o más interiores.
MultiPoint	0	Agrupación no conexa de puntos independientes.
MultiLineString	1	Conjunto estructurado de cadenas de líneas.

Tipo geométrico	Dimensión topológica	Descripción formal OGC
MultiPolygon	2	Conjunto conexo o disyunto de polígonos.
GeometryCollection	Variable	Agrupación heterogénea arbitraria de entidades geométricas.

Serialización y formatos estándar: WKT y WKB

Para garantizar el intercambio de geometrías entre sistemas heterogéneos, el estándar OGC define dos formatos de interoperabilidad:

Formato Well-Known Text (WKT)

WKT es una codificación en texto plano legible por humanos que define la estructura del tipo geométrico y sus secuencias de coordenadas:

- POINT(1000000 1000000)
- LINESTRING(1000000 1000000, 1050000 1050000, 1100000 1020000)
- POLYGON((1000000 1000000, 1100000 1000000, 1100000 1100000, 1000000 1100000, 1000000 1000000))

En superficies poligonales, el primer anillo define el perímetro exterior, mientras que los anillos sucesivos delimitan perforaciones interiores:

```
POLYGON((0 0, 10 0, 10 10, 0 10, 0 0), (2 2, 4 2, 4 4, 2 4, 2 2))
```

PostGIS extiende el estándar WKT incorporando componentes tridimensionales y de medición lineal: POINT Z, POINT M y POINT ZM.

Formato Well-Known Binary (WKB) y EWKB

WKB es una representación binaria estructurada en secuencias de bytes (*byte arrays*), optimizada para la transferencia en red y el almacenamiento interno en páginas de disco:

- Incorpora indicadores de ordenamiento de bytes (*Endianness*: Big-Endian o Little-Endian).
- Define identificadores numéricos que codifican el tipo de geometría.
- Preserva la precisión numérica en coma flotante de doble precisión (IEEE 754) para cada coordenada.

PostGIS implementa el formato extendido **EWKB** (*Extended Well-Known Binary*), el cual embebe directamente el identificador del sistema de referencia espacial (SRID) en la cabecera binaria del objeto geométrico.

Catálogos de metadatos espaciales

En un motor relacional espacial, el clúster almacena la información cartográfica respetando las reglas de metadatos del estándar OGC a través de dos componentes centrales:

- **spatial_ref_sys**: Diccionario canónico de sistemas de referencia de coordenadas (CRS). Define las proyecciones, elipsoides y datums que el motor es capaz de interpretar y transformar.
- **geometry_columns**: Inventario dinámico de capas espaciales. Registra las tablas y columnas geométricas instanciadas físicamente en los esquemas de la base de datos.

El catálogo spatial_ref_sys

El catálogo `spatial_ref_sys` es la tabla canónica donde residen las definiciones de todos los sistemas de referencia de coordenadas soportados por el motor:

```
export PGPASSWORD="${LOCAL_PG_PASSWORD}"

query_cols="
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'spatial_ref_sys';
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_cols}"
```

column_name		data_type
sr_id		integer
auth_name		character varying
auth_sr_id		integer
sridtext		character varying
proj4text		character varying

(5 rows)

Sus columnas clave comprenden:

- **srid**: Identificador entero del sistema de referencia espacial (habitualmente alineado con el estándar internacional EPSG).
- **auth_name**: Nombre del organismo emisor de la norma (comúnmente 'EPSG').
- **auth_srid**: Código numérico asignado por la autoridad cartográfica.
- **srtext**: Definición formal en formato WKT CRS (*Well-Known Text of Spatial Reference Systems*), requerida por clientes SIG y bibliotecas de procesamiento geométrico.
- **proj4text**: Cadena de inicialización de parámetros empleada por el motor de transformación cartográfica PROJ.

Al consultar las definiciones preinstaladas para Colombia, el motor lista los marcos geodésicos históricos y las fajas Gauss-Krüger tradicionales:

```
export PGPASSWORD="${LOCAL_PG_PASSWORD}"

query_sistemas_colombia="
SELECT
    srid,
    auth_name,
    split_part(srtext, '\", 2) AS nombre_sistema
FROM spatial_ref_sys
WHERE srtext ILIKE '%Colombia%' OR srtext ILIKE '%MAGNA-SIRGAS%'
ORDER BY srid;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_sistemas_colombia}"
```

srid	auth_name	nombre_sistema
3114	EPSG	MAGNA-SIRGAS / Colombia Far West zone
3115	EPSG	MAGNA-SIRGAS / Colombia West zone
3116	EPSG	MAGNA-SIRGAS / Colombia Bogota zone
3117	EPSG	MAGNA-SIRGAS / Colombia East Central zone
3118	EPSG	MAGNA-SIRGAS / Colombia East zone
4686	EPSG	MAGNA-SIRGAS
4996	EPSG	MAGNA-SIRGAS
4997	EPSG	MAGNA-SIRGAS
9377	EPSG	MAGNA-SIRGAS / Origen-Nacional
21891	EPSG	Bogota 1975 / Colombia West zone (deprecated)
21892	EPSG	Bogota 1975 / Colombia Bogota zone (deprecated)

21893		EPSG		Bogota 1975 / Colombia East Central zone (deprecated)
21894		EPSG		Bogota 1975 / Colombia East (deprecated)
21896		EPSG		Bogota 1975 / Colombia West zone
21897		EPSG		Bogota 1975 / Colombia Bogota zone
21898		EPSG		Bogota 1975 / Colombia East Central zone
21899		EPSG		Bogota 1975 / Colombia East
102231		ESRI		Colombia_West_West_Zone
102233		ESRI		MAGNA_Ciudad_Bogota

(19 rows)

 Ausencia del código EPSG 9377 en volcados iniciales de spatial_ref_sys

La tabla `spatial_ref_sys` es una tabla relacional física en disco poblada a partir de los scripts iniciales de PostGIS o volcados históricos (como `init_unal_spatialdb.sql`). A diferencia del software de escritorio moderno o la biblioteca PROJ (que actualizan sus parámetros en cada compilación contra `proj.db`), el catálogo relacional en PostgreSQL es estático y no se actualiza de forma automática.

El sistema oficial de Colombia con Origen Nacional Único fue adoptado por el Instituto Geográfico Agustín Codazzi mediante la Resolución IGAC 471 de 2020 e indexado en el registro internacional bajo el código **EPSG:9377**. Si el clúster se inicializó con un esquema previo a dicha norma, la tupla no existirá en disco.

Aunque el tipado declarativo de PostgreSQL permite crear columnas con `geometry(Point, 9377)` sin validar la tabla, cualquier operación analítica de cálculo o reproyección cartográfica (`ST_Transform`) fallará en tiempo de ejecución si el registro no reside en `spatial_ref_sys`.

Como labor fundamental de administración de bases de datos (DBA), el registro oficial debe ser incorporado al catálogo. **Esta instrucción DML solo requiere ejecutarse una única vez en la base de datos**, pues la cláusula `ON CONFLICT` garantiza su persistencia permanente sin duplicación:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_insert_9377="
INSERT INTO spatial_ref_sys (srid, auth_name, auth_srid, srtext, proj4text)
VALUES (
    9377,
    'EPSG',
    9377,
    'PROJCS[\"MAGNA-SIRGAS / Origen-Nacional\",GEOGCS[\"MAGNA-SIRGAS\",DATUM[\"Marco_Geocentri\",
    '+proj=tmerc +lat_0=4 +lon_0=-73 +k=0.9992 +x_0=5000000 +y_0=2000000 +ellps=GRS80 +towgs84=0,0,0,0,0,0,0
    )
```

```
ON CONFLICT (srid) DO UPDATE
SET srtext = EXCLUDED.srtext, proj4text = EXCLUDED.proj4text;
"
```

```
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-c "${query_insert_9377}"
```

```
INSERT 0 1
```

Mediante la cláusula `ON CONFLICT (srid) DO UPDATE`, la sentencia formaliza una operación atómica de tipo **UPSERT**, confiriendo **idempotencia** a la ejecución:

- **Primera compilación:** Al no existir el identificador 9377, el motor ejecuta la inserción estándar (`INSERT`) en el catálogo físico.
- **Compilaciones sucesivas:** El motor detecta la colisión sobre el índice de la clave primaria (`srid`) y, en lugar de abortar la transacción por infracción de unicidad (`duplicate key value violates unique constraint`), actualiza los parámetros sobre la tupla pre-existente.

Este diseño garantiza que los ciclos continuos de renderizado en Quarto mantengan el catálogo en un estado consistente y libre de registros redundantes, sin interrumpir el flujo de publicación del texto técnico.

Una vez registrada la tupla en disco, la inspección detallada de los sistemas cartográficos de referencia para el país expone sus parámetros elipsoidales y de proyección:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"
```

```
query_colombia="
SELECT srid, auth_name, auth_srid, srtext
FROM spatial_ref_sys
WHERE srid IN (4326, 3116, 9377)
ORDER BY srid;
"
```

```
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-x -P format=wrapped -P columns=90 \
-c "${query_colombia}"
```

```
-----
-[ RECORD 1 ]-----
srid          | 3116
auth_name     | EPSG
```

```

auth_srid | 3116
srtext    | PROJCS["MAGNA-SIRGAS / Colombia Bogota zone",GEOGCS["MAGNA-SIRGAS",DATUM["Mar.
|.co_Geocentrico_Nacional_de_Referencia",SPHEROID["GRS 1980",6378137,298.257222.
|.101,AUTHORITY["EPSG","7019"]],TOWGS84[0,0,0,0,0,0],AUTHORITY["EPSG","6686"].
|.],PRIMEM["Greenwich",0,AUTHORITY["EPSG","8901"]],UNIT["degree",0.017453292519.
|.9433,AUTHORITY["EPSG","9122"]],AUTHORITY["EPSG","4686"]],PROJECTION["Transver.
|.se_Mercator"],PARAMETER["latitude_of_origin",4.596200416666666],PARAMETER["ce.
|.ntral_meridian",-74.07750791666666],PARAMETER["scale_factor",1],PARAMETER["fa.
|.lse_easting",1000000],PARAMETER["false_northing",1000000],UNIT["metre",1,AUTH.
|.ORITY["EPSG","9001"]],AUTHORITY["EPSG","3116"]]

-[ RECORD 2 ]-----
srid      | 4326
auth_name | EPSG
auth_srid | 4326
srtext    | GEOGCS["WGS 84",DATUM["WGS_1984",SPHEROID["WGS 84",6378137,298.257223563,AUTH.
|.ORITY["EPSG","7030"]],AUTHORITY["EPSG","6326"]],PRIMEM["Greenwich",0,AUTHORIT.
|.Y["EPSG","8901"]],UNIT["degree",0.0174532925199433,AUTHORITY["EPSG","9122"]],.
|.AUTHORITY["EPSG","4326"]]

-[ RECORD 3 ]-----
srid      | 9377
auth_name | EPSG
auth_srid | 9377
srtext    | PROJCS["MAGNA-SIRGAS / Origen-Nacional",GEOGCS["MAGNA-SIRGAS",DATUM["Marco_Ge.
|.ocentrico_Nacional_de_Referencia",SPHEROID["GRS 1980",6378137,298.257222101,A.
|.UTHORITY["EPSG","7019"]],AUTHORITY["EPSG","6686"]],PRIMEM["Greenwich",0,AUTHO.
|.RITY["EPSG","8901"]],UNIT["degree",0.0174532925199433,AUTHORITY["EPSG","9122".
|.]],AUTHORITY["EPSG","4686"]],PROJECTION["Transverse_Mercator"],PARAMETER["lat.
|.itude_of_origin",4],PARAMETER["central_meridian",-73],PARAMETER["scale_factor.
|.","0.9992],PARAMETER["false_easting",5000000],PARAMETER["false_northing",20000.
|.00],UNIT["metre",1,AUTHORITY["EPSG","9001"]],AUTHORITY["EPSG","9377"]]

```

Entre estos identificadores fundamentales destacan:

- **SRID 4326:** Sistema geográfico global WGS 84 (coordenadas angulares en grados decimales de latitud y longitud).
- **SRID 3116:** Proyección MAGNA-SIRGAS / Colombia Bogotá (proyección conforme Gauss-Krüger sobre meridiano central $74^{\circ}04'39.0285''$ W, con falso este de 1 000 000 m y falso norte de 1 000 000 m).
- **SRID 9377:** Proyección MAGNA-SIRGAS / Origen Nacional Único (sistema oficial conforme a la Resolución IGAC 471 de 2020, con origen en 4° N, 73° W, factor de escala $k_0 = 0.9992$, falso este de 5 000 000 m y falso norte de 2 000 000 m).

La vista de catálogo geometry_columns

El estándar OGC estipula que toda tabla relacional que incorpore una o más columnas espaciales debe estar documentada de manera uniforme en el catálogo `geometry_columns`.

En versiones contemporáneas de PostGIS, `geometry_columns` no es una tabla física estática que deba poblarse manualmente, sino una vista del sistema construida dinámicamente mediante introspección sobre el catálogo de PostgreSQL (`pg_class`, `pg_attribute`, `pg_type`). Esta vista extrae las restricciones declarativas y el tipado estricto (*typmod*, por ejemplo `geometry(Point, 9377)`):

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_geom_cols="
SELECT f_table_schema, f_table_name, f_geometry_column, coord_dimension, srid, type
FROM geometry_columns
LIMIT 10;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_geom_cols}"
```

f_table_schema	f_table_name	f_geometry_column	coord_dimension	srid	type
base750	hidro_lineas	the_geom	2	21892	GEOMETRY
base750	vias_lineas	the_geom	2	21892	GEOMETRY
base750	vias_puntos	the_geom	2	21892	ST_Point
base750	vias_texto	the_geom	2	21892	ST_Point
divpol	departamentos	the_geom	2	21892	GEOMETRY
divpol	capa_departame. .ntos	the_geom	2	0	GEOMETRY
divpol	municipios	the_geom	2	21892	GEOMETRY
divpol	capa_municipio. .s	the_geom	2	0	GEOMETRY
public	infraestructur. .a_referencia	geom	2	9377	POINT
(9 rows)					

Los campos descriptivos obligatorios según la norma OGC son:

- `f_table_schema`: Esquema relacional donde reside la tabla (ej. `public`, `base750`, `divpol`).

- **f_table_name**: Nombre de la tabla que contiene la capa espacial.
- **f_geometry_column**: Nombre de la columna que almacena los objetos espaciales (convencionalmente **geom** o **the_geom**).
- **coord_dimension**: Dimensión de las coordenadas almacenadas (2 para *XY*, 3 para *XYZ/XYM*, o 4 para *XYZM*).
- **srid**: Identificador entero del sistema de referencia espacial asociado.
- **type**: Restricción del tipo geométrico (**POINT**, **LINESTRING**, **POLYGON**, o **GEOMETRY** si admite tipos mixtos).

Para realizar una auditoría integral de la base de datos pedagógica **sig_db_unal**, se combina la vista **geometry_columns** con la tabla **spatial_ref_sys**. Esta consulta consolida los marcos de referencia en uso real, identificando las capas cartográficas existentes por esquema y detectando capas sin proyección asignada (**srid**: 0):

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_crs_en_uso="
SELECT
    g.srid,
    COALESCE(s.auth_name, 'LOCAL') AS autoridad,
    COALESCE(split_part(s.srtext, '\'', 2), 'Sin CRS asignado (Plano / Indefinido)') AS descripcion_crs,
    count(*) AS numero_de_capas,
    string_agg(g.f_table_schema || '.' || g.f_table_name, ', ' ) AS capas_asociadas
FROM geometry_columns g
LEFT JOIN spatial_ref_sys s ON g.srid = s.srid
GROUP BY g.srid, s.auth_name, s.srtext
ORDER BY g.srid;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-x -P format=wrapped -P columns=90 \
-c "${query_crs_en_uso}"
```

```
-[ RECORD 1 ]-----+-----
srid          | 0
autoridad     | LOCAL
descripcion_crs | Sin CRS asignado (Plano / Indefinido)
numero_de_capas | 2
capas_asociadas | divpol.capa_departamentos, divpol.capa_municipios
-[ RECORD 2 ]-----+-----
srid          | 9377
autoridad     | EPSG
```

```

descripcion_crs | MAGNA-SIRGAS / Origen-Nacional
numero_de_capas | 1
capas_asociadas | public.infraestructura_referencia
-[ RECORD 3 ]---+-----
srid            | 21892
autoridad       | EPSG
descripcion_crs | Bogota 1975 / Colombia Bogota zone (deprecated)
numero_de_capas | 6
capas_asociadas | base750.vias_texto, base750.hidro_lineas, divpol.municipios, divpol.dep.
                  |.artamentos, base750.vias_lineas, base750.vias_puntos

```

Asignación de identificador espacial en las vistas del esquema

Al examinar los resultados de la consulta cruzada anterior, se constata que las relaciones `capa_departamentos` y `capa_municipios` del esquema `divpol` reportan `srid: 0`. Esto ocurre porque ambos objetos son vistas relacionales (consultas almacenadas) en las que la columna geométrica fue proyectada sin un identificador de referencia explícito, lo que ocasiona que los clientes SIG interpreten la proyección como indefinida.

Aunque los fundamentos formales de DDL y DML para el diseño avanzado de vistas espaciales se estudiarán detalladamente en capítulos posteriores, es indispensable regularizar el catálogo desde esta primera sesión. La solución consiste en redefinir ambas vistas forzando la conversión explícita de la columna geométrica al sistema de coordenadas de la cartografía base del curso (EPSG:21892, MAGNA-SIRGAS / Colombia Bogotá):

```

export PGPASSWORD="${LOCAL_PGPASSWORD}"

# Redefinición de vistas con asignación explícita de SRID 21892
query_recreacion_vistas="
DROP VIEW IF EXISTS divpol.capa_departamentos;
CREATE VIEW divpol.capa_departamentos AS
SELECT
    d.gid,
    d.gid AS oid,
    d.dep_id AS id_dane,
    d.dep_nombre AS departamento,
    d.the_geom::geometry(Geometry, 21892) AS the_geom
FROM divpol.departamentos d
WHERE d.the_geom IS NOT NULL
ORDER BY d.dep_id;

DROP VIEW IF EXISTS divpol.capa_municipios;

```

```

CREATE VIEW divpol.capa_municipios AS
SELECT
    m.gid,
    m.gid AS oid,
    m.mun_id AS id_dane,
    d.dep_id AS dep_codigo_dane,
    initcap(d.dep_nombre::text) AS departamento,
    m.mun_nombre AS municipio,
    m.the_geom::geometry(Geometry, 21892) AS the_geom
FROM divpol.municipios m
LEFT JOIN divpol.departamentos d ON d.dep_id::text = m.dep_id::text
WHERE m.the_geom IS NOT NULL
ORDER BY m.mun_id;
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-c "${query_recreacion_vistas}"

```

```

DROP VIEW
CREATE VIEW
DROP VIEW
CREATE VIEW

```

Tras la ejecución del bloque anterior, se audita nuevamente el catálogo `geometry_columns` para validar la correcta inscripción del identificador espacial:

```

export PGPASSWORD="${LOCAL_PGPASSWORD}"

query_verif_vistas="
SELECT
    f_table_schema,
    f_table_name,
    f_geometry_column,
    coord_dimension,
    srid,
    type
FROM geometry_columns
WHERE f_table_schema = 'divpol'
    AND f_table_name IN ('capa_departamentos', 'capa_municipios');
"

```

```
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_verif_vistas}"
```

f_table_schema	f_table_name	f_geometry_column	coord_dimension	srid	type
divpol	capa_departame.	the_geom	2	21892	GEOMETRY
	.ntos				
divpol	capa_municipio.	the_geom	2	21892	GEOMETRY
	.s				

(2 rows)

El catálogo reporta ahora el identificador 21892, garantizando la interoperabilidad con clientes de escritorio y servicios web.

Persistencia del catálogo mediante respaldo

Tras incorporar el sistema EPSG:9377 en el catálogo `spatial_ref_sys` y corregir el tipado de las vistas del esquema `divpol`, el catálogo relacional alcanza un estado consistente con el estándar OGC.

Para preservar estas modificaciones frente a eventuales destrucciones del volumen persistente, fallos del contenedor o reinicializaciones del clúster local, se recomienda generar un nuevo respaldo de la base de datos `sig_db_unal` aplicando los procedimientos documentados en la [?@sec-respaldo-recuperacion-bd](#). Disponer de un archivo de volcado actualizado (en formato SQL plano o binario personalizado `.dump`) permite reconstruir la base de datos con las definiciones espaciales saneadas, eliminando la necesidad de reejecutar manualmente las sentencias de remediación en futuras restauraciones.

Verificación práctica en la consola integrada

A continuación se demuestra la secuencia operativa para comprobar la disponibilidad de la extensión espacial, aprovisionar una tabla pedagógica con tipado geométrico estricto, insertar una entidad mediante sintaxis WKT y auditar su catalogación automática:

```
export PGPASSWORD="${LOCAL_PGPASSWORD}"

# 1. Inspección de extensiones espaciales activas
psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "SELECT extname, extversion FROM pg_extension WHERE extname = 'postgis';"
```

```
# 2. Creación e inserción pedagógica (con limpieza previa para asegurar idempotencia)
query_ddl="
DROP TABLE IF EXISTS public.infraestructura_referencia;

CREATE TABLE public.infraestructura_referencia (
    id serial PRIMARY KEY,
    codigo varchar(10) NOT NULL UNIQUE,
    descripcion text,
    geom geometry(Point, 9377) NOT NULL
);

INSERT INTO public.infraestructura_referencia (codigo, descripcion, geom)
VALUES (
    'EST-001',
    'Estación de monitoreo central',
    ST_GeomFromText('POINT(5000000 2000000)', 9377)
);
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-c "${query_ddl}"

# 3. Auditoría de registro automático en geometry_columns
query_audit="
SELECT f_table_schema, f_table_name, f_geometry_column, coord_dimension, srid, type
FROM geometry_columns
WHERE f_table_name = 'infraestructura_referencia';
"

psql -h ${LOCAL_PGHOST} -p ${LOCAL_PGPORT} -U ${LOCAL_PGUSER} -d ${LOCAL_PGDATABASE} \
-P format=wrapped -P columns=90 \
-c "${query_audit}"
```

```

  extname | extversion
-----+-----
 postgres | 3.6.2
(1 row)
```

```
DROP TABLE
CREATE TABLE
INSERT 0 1
```

f_table_schema	f_table_name	f_geometry_column	coord_dimension	srid	type
public	infraestructura_ref.	geom	2	9377	POINT
	.erencia				

(1 row)

La consulta a `geometry_columns` reporta de forma automática la dimensión 2, el SRID 9377 y el tipo de dato `POINT`, validando la interoperabilidad del clúster con clientes SIG externos.